TRAINING • TECHNIZER INDIA

GitHub Copilot

Architecture, Models & Developer Capabilities

Facilitator

Manish Sharma

Technizer India

GPT-5

Claude Opus 4.5

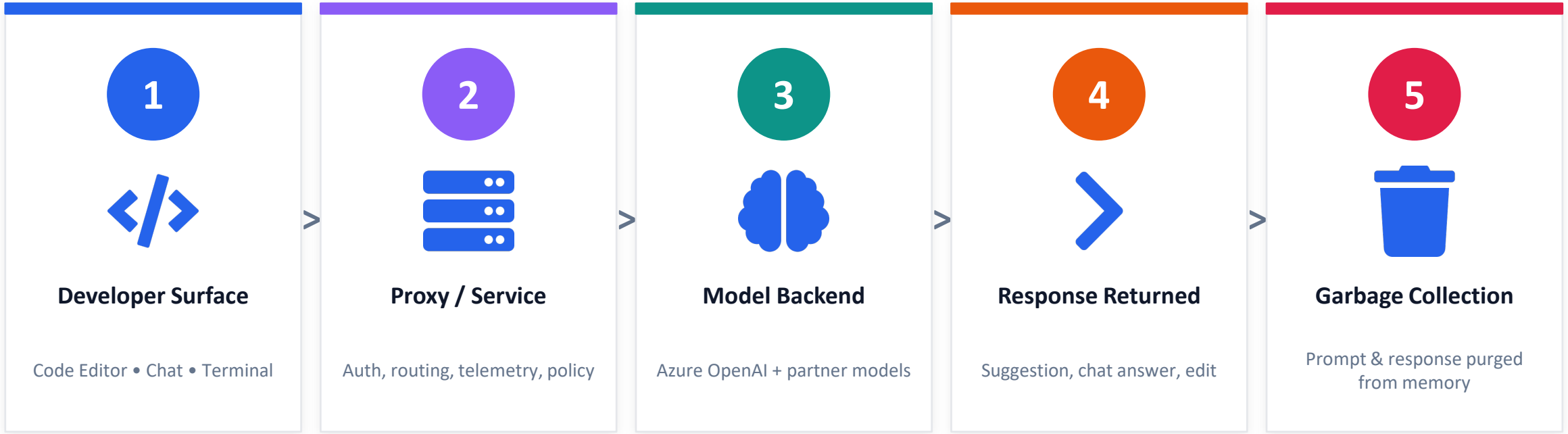
Gemini 3 Pro

Grok Code Fast

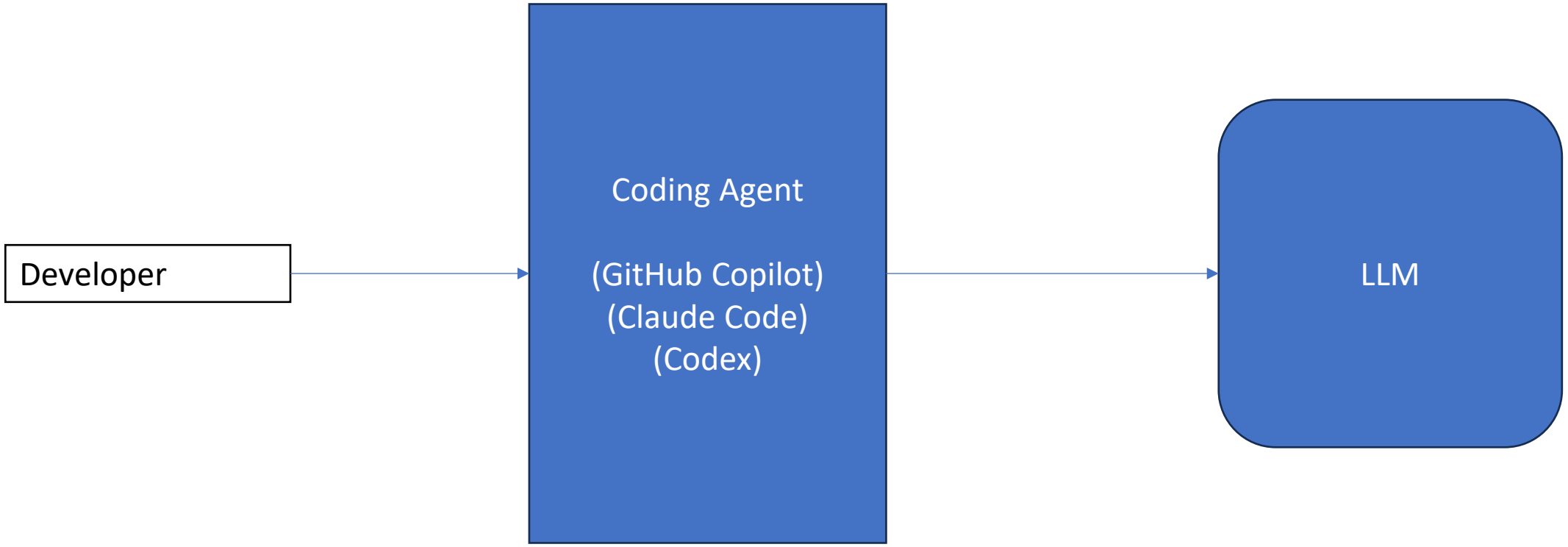
HOW IT WORKS

Copilot Request Flow — From Editor to Model

A request from your IDE travels through a proxy, hits the model, returns a response, and is then purged from memory.



Privacy by design — prompts and code suggestions are not retained after the request completes.


Developer
Coding Agent
(GitHub Copilot)
(Claude Code)
(Codex)
LLM

ARCHITECTURE

The Copilot Stack — Layered View

Each layer has a single responsibility: the frontend collects context, the orchestrator builds a safe prompt, the foundation layer generates the answer.

Copilot Frontend

Editor surface, chat UI, plugin extensibility

Orchestration Layer

Prompt filtering • Metaprompt • Grounding • Plugin execution

Safety & Grounding

Content policy, retrieval, response filtering

Foundation Models

GPT • Claude • Gemini • Grok — routed by task

FOUNDATION MODELS

Routed per task & plan

OpenAI
GPT-5 • GPT-5.1 • GPT-5-Codex

Anthropic
Claude Opus 4.5 • Sonnet 4.5 • Haiku 4.5

Google
Gemini 3 Pro • Gemini 2.5 Pro

xAI
Grok Code Fast 1

PLANS & ACCESS

Model Availability Across Copilot Plans

✓ Included — Not available • **Family colour** *indicates provider*

Family	Model	Free	Pro	Pro+	Business	Enterprise
Anthropic	Claude Haiku 4.5	✓	✓	✓	✓	✓
Anthropic	Claude Sonnet 4	—	✓	✓	✓	✓
Anthropic	Claude Sonnet 4.5	—	✓	✓	✓	✓
Anthropic	Claude Opus 4.1	—	—	✓	—	✓
Anthropic	Claude Opus 4.5	—	✓	✓	✓	✓
OpenAI	GPT-4.1	✓	✓	✓	✓	✓
OpenAI	GPT-5 mini	—	✓	✓	✓	✓
OpenAI	GPT-5	—	✓	✓	✓	✓
OpenAI	GPT-5-Codex	—	✓	✓	✓	✓
OpenAI	GPT-5.1	—	✓	✓	✓	✓
OpenAI	GPT-5.1-Codex	—	✓	✓	✓	✓
OpenAI	GPT-5.1-Codex-Mini	—	✓	✓	✓	✓
OpenAI	GPT-5.1-Codex-Max	—	✓	✓	✓	✓
Google	Gemini 2.5 Pro	—	✓	✓	✓	✓
Google	Gemini 3 Pro	—	✓	✓	✓	✓
xAI	Grok Code Fast 1	—	✓	✓	✓	✓
GitHub	Raptor mini	✓	✓	✓	—	—

MODEL SELECTION

Model vs Task — At a Glance

DEEP REASONING

GPT-5 & Claude Opus

Best for debugging and architecture-level thinking.

SPEED

Haiku 4.5 & Raptor mini

Best for speed and repetitive developer tasks.

BALANCED

Sonnet 4 / 4.5

Best balance for agents and daily dev workflows.

RESEARCH

Gemini 2.5 Pro

Ideal for research and complex debugging.

LOW LATENCY

Grok Code Fast 1

Optimised for low-latency, fast coding.

REPAIR

Qwen2.5

Good for reasoning and code repair.

DEEP DIVE

Detailed Model Capabilities

Model	Task Area	Excels At	Capabilities
GPT-5-Codex	General-purpose coding	Fast, accurate completions and explanations	*Agent mode*
GPT-5 mini	General-purpose coding	Fast, accurate completions and explanations	*Agent, reasoning, vision*
GPT-5	Deep reasoning & debugging	Multi-step problem solving, architecture analysis	*Reasoning*
Claude Haiku 4.5	Lightweight tasks	Fast, reliable answers to simple questions	*Agent mode*
Claude Sonnet 4.5	Coding & agent tasks	Complex problem-solving, sophisticated reasoning	*Agent mode*
Claude Opus 4.1	Deep reasoning & debugging	Complex problem-solving, sophisticated reasoning	*Reasoning, vision*
Claude Sonnet 4	Deep reasoning & debugging	Performance and practicality, balanced for coding	*Agent mode, vision*
Gemini 2.5 Pro	Deep reasoning & debugging	Complex code gen, debugging, research workflows	*Reasoning, vision*
Grok Code Fast 1	General-purpose coding	Fast, accurate completions and explanations	*Agent mode*
Qwen2.5	General-purpose coding	Code generation, reasoning, code repair	*Reasoning*
Raptor mini	General-purpose coding	Fast, accurate code	*Agent mode*

The Prompt Formula

Role	Who is Copilot acting as? "Acting as a senior TypeScript engineer..."
Task	The one verb-driven thing you want done.
Context	What does it need to know? The framework, the file's purpose, the data shape.
Constraints	What must it not do, must use, must avoid?
Output Format	Code only? Code + explanation? A specific function signature?

Heads up: *Not every prompt needs all five. Inline completions lean on the file's context — a one-line comment may be enough. Chat prompts, especially for non-trivial tasks, should use the full formula.*

Variable	Description
#block	Includes the current block of code in the prompt.
#class	Includes the current class in the prompt.
#comment	Includes the current comment in the prompt.
#file	Includes the current file's content in the prompt.
#function	Includes the current function or method in the prompt.
#line	Includes the current line of code in the prompt.
#path	Includes the file path in the prompt.
#project	Includes the project context in the prompt.
#selection	Includes the currently selected text in the prompt.
#sym	Includes the current symbol in the prompt.

Chat variables

- Use chat variables to include specific context in your prompt. To use a chat variable, type # in the chat prompt box, followed by a chat variable.

Variable	Description
@azure	Has context about Azure services and how to use, deploy and manage them. Use @azure when you want help with Azure. The @azure chat participant is currently in public preview and is subject to change.
@github	Allows you to use GitHub-specific Copilot skills. See Asking GitHub Copilot questions in your IDE.
@terminal	Has context about the Visual Studio Code terminal shell and its contents. Use @terminal when you want help creating or debugging terminal commands.
@vscode	Has context about Visual Studio Code commands and features. Use @vscode when you want help with Visual Studio Code.
@workspace	Has context about the code in your workspace. Use @workspace when you want Copilot to consider the structure of your project, how different parts of your code interact, or design patterns in your project.

Chat participants

- Chat participants are like domain experts who have a specialty that they can help you with.
- You can specify a chat participant by typing @ in the chat prompt box, followed by a chat participant name.

PRODUCTIVITY

Slash Commands — Quick Actions in Chat

/

Slash commands

Trigger built-in Copilot actions directly from the chat. Type / followed by the command to run a specific task on your active editor or workspace.

Command	Description
`/clear`	Start a new chat session.
`/explain`	Explain how the code in your active editor works.
`/fix`	Propose a fix for problems in the selected code.
`/fixTestFailure`	Find and fix a failing test.
`/help`	Quick reference and basics of using GitHub Copilot.
`/new`	Create a new project.
`/tests`	Generate unit tests for the selected code.

CONTEXT INJECTION

Chat Variables — Bring Precise Context

#

Chat variables

Reference specific scope in your prompt. Type # in the chat box and pick a variable to inject exactly the right context — no copy-paste.

Variable	Description
#block	Includes the current block of code in the prompt.
#class	Includes the current class in the prompt.
#comment	Includes the current comment in the prompt.
#file	Includes the current file's content in the prompt.
#function	Includes the current function or method in the prompt.
#line	Includes the current line of code in the prompt.
#path	Includes the file path in the prompt.
#project	Includes the project context in the prompt.
#selection	Includes the currently selected text in the prompt.
#sym	Includes the current symbol in the prompt.

DOMAIN EXPERTS

Chat Participants — Specialised Assistants

Chat participants

Domain experts you can summon by typing @ in the chat. Each participant knows a specific area and pulls the right context automatically.

Participant	Description
`@azure`	Context on Azure services — use to deploy and manage Azure resources. (Public preview)
`@github`	Use GitHub-specific Copilot skills directly from the IDE.
`@terminal`	Context about the VS Code terminal — for creating or debugging shell commands.
`@vscode`	Context about VS Code commands and features.
`@workspace`	Context about the structure of your project, how parts interact, and design patterns.

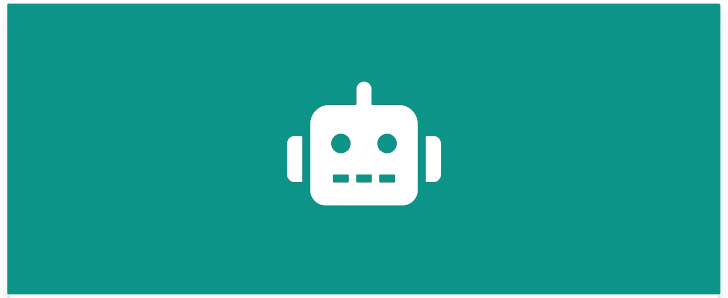
Copilot Chat — Three Modes, Three Intents

Pick the mode that matches what you want Copilot to do — answer, act, or plan.

ANSWER

Ask mode

Get answers to coding questions and code suggestions inline.



EXECUTE

Agent mode

Autonomously accomplish a set task with tools and iteration.

DESIGN

Plan mode

Produce detailed implementation plans before any code is written.

Rule of thumb: Ask for understanding • Plan before complex work • Agent for autonomous execution.

```
[ROLE]
Act as a [Senior/Expert Tier] [Specialization] Engineer.

[TASK]
I need you to [High-Level Goal / Feature Name] in our application.

[CONTEXT]
- Core Logic/Routes: [File Paths]
- Models/Database: [File Paths]
- Existing Configuration: [File Paths]

[CONSTRAINTS]
- Tech Stack Limits: Do not use [Libraries/Frameworks]. Use [Existing Packages].
- Security/Business Rules: Enforce [Limits/Validations/Rules].
- Scope: Only modify files related to [Feature], do not refactor outside this sc

[EXECUTION STEPS]
1. Database: Update [File] to include [Fields/Types].
2. Logic: Create [File Path] and implement [Specific Algorithm/Logic].
3. Routing: Expose the feature at [Endpoint/Page URL].

[VERIFICATION]
- Run [Local Test Script / Simulation Command] to test cases: [Case A, Case B].
- Run [Build/Lint Command] to ensure zero compilation or styling errors.
```

LET'S CONTINUE THE CONVERSATION

Thank you.

Questions, feedback, or follow-up training — reach out any time.

Manish Sharma

Technizer India

manishvbn@gmail.com

manish.sharma@technizerindia.com

WhatsApp • +91 98220 25942

LinkedIn • Manish Sharma Technizer

GitHub

Copilot

Corporate Training

2026